

The Master Architecture Roadmap

Phase 1: Data Engineering & State Space Parsing

Before writing any predictive algorithms, you must transform raw, unstructured event coordinate data into a discrete, structured timeline of state transitions.

Task 1.1: Web Scraping Pipeline

Finalize `url_harvester.py` and `epl_scraper.py` to compile match event logs into local raw files (JSON/CSV format).

Task 1.2: Spatial & Possession Discretisation

Write a script to parse raw coordinates and assign every event to one of your 5 spatial zones. Map these to your 10 active states based on which team has possession, plus your 2 terminal absorbing states: Goal (Home) and Goal (Away).

Task 1.3: Transition Matrix Assembly

Calculate the elapsed time (Δt) between consecutive events. Structure an event log parser that outputs an array of tuples: (`current_state`, `next_state`, `time_spent`).

Phase 2: Statistical Prior & Matrix Calibration

In this phase, you establish the baseline laws of football physics for the entire tournament before skewing them for specific matchups.

Task 2.1: The Global Baseline Prior (Q_{global})

Aggregate all transition counts and time-spent metrics across every processed match in your database. Compute the global transition rate matrix using maximum likelihood estimation:

$$\lambda_{ij} = \frac{n_{ij}}{T_i}$$

Where n_{ij} is the total count of transitions from state i to state j , and T_i is the total cumulative time the ball spent in state i .

Task 2.2: Team-Specific Bayesian Updating (Q_{team})

Implement a conjugate updating function to blend a team's sparse individual match history with the dense global prior. Use prior pseudo-counts to ensure the matrix remains stable even for teams with minimal historical data.

Phase 3: The Matchup Layer (Pre-Match Forecasting)

Here, you convert your standalone team profiles into a combined, competitive environment adjusted for team strength.

Task 3.1: Proportional Hazards Scaling Engine

Build a mathematical modifier module that ingests the pre-match Elo rating differential (ΔElo).

Hardcode the structural “sentiment” tags for every valid transition path. Positive actions (progression) get inflated by a positive differential; negative actions (turnovers) get suppressed, governed by a calibrated sensitivity constant β .

Task 3.2: GPU-Accelerated Monte Carlo Simulator

Vectorize your simulation loop using PyTorch or NumPy array operations to fully leverage hardware acceleration.

Program the random walk generator: place the ball at the midfield state at $t = 0$, sample next-state transitions using the probabilities derived from the matrix, track goal events, and terminate at $t = 90$. Run 10,000 parallel iterations to calculate $P_{CTMC}(W, D, L)$.

Task 3.3: Pre-Match k-NN Engine

Extract static macro-features for historical matches (e.g., pre-match Elo spreads, historical average possession splits, tournament stage).

Implement a standard k-NN classifier to output empirical match outcome probabilities $P_{kNN}(W, D, L)$ based on historical baseline conditions.

Phase 4: In-Play Live Pricing Engine

This phase transitions the model from static pre-match forecasting to a highly reactive, real-time data streaming pipeline.

Task 4.1: The In-Game Bayesian Refresher

Create a live data pipeline that tracks events occurring during the live match.

At any minute t , append the live counts directly into the matrix calculation numerator and denominator, forcing the initial Elo-scaled matrix to bend toward the live, empirical reality of the pitch.

Task 4.2: Macro-Trajectory Tracking (Live k-NN)

Calculate rolling 5-minute match metrics during the live fixture (e.g., Expected Threat generated, defensive box entries, pass completion deltas).

Query your historical database for matches that exhibited the most similar rolling momentum curves up to minute t .

Task 4.3: Dynamic Late-Fusion Ensembling

Implement your time-decay blending function:

$$P_{final} = \alpha(t)P_{CTMC} + (1 - \alpha(t))P_{kNN}$$

Calibrate $\alpha(t)$ to start high (relying on structural matrix physics) and decay linearly or exponentially as $t \rightarrow 90$ (relying on historical game-state management).

Phase 5: Backtesting, Validation & Calibration

The final step ensures your model generates mathematically sound probabilities that can withstand the efficiency of real betting markets.

Task 5.1: Historical Holdout Backtesting

Run your entire pipeline retroactively against a holdout set of matches. Generate live probability curves for every minute of those games.

Task 5.2: Probability Calibration Assessment

Compute the Brier Score and Log Loss across all simulated time steps.
Plot a calibration curve to ensure that when your model claims a team has a 70% chance of winning at the 60th minute, they actually win exactly 70% of the time.